CSE 302 - Operating Systems: Banker's Algorithm Simulator in C

Name: Aditya Kumar

Roll number (registration no): AP24110010333

BTech CSE - C

Question:

Write a C program that simulates the Banker's Algorithm for N processes and M resource types. It reads Available, Allocation and Max from standard input and answers two kinds of query: a safety check (print a safe sequence or UNSAFE) and a resource request (GRANTED, or DENIED with a reason, rolling the state back when the request would lead to an unsafe state).

Design summary:

1. State is kept in dynamically allocated matrices (Allocation, Max, Need = Max - Allocation) plus an Available vector, created with alloc_matrix() and released with free_matrix(); main() has a single cleanup label, so every exit path frees everything.

2. Input is read token by token with strtol(), so wrong token counts, non-integers, negative or out-of-range values, Allocation > Max, unknown query letters, bad process indexes and trailing tokens are all rejected before anything is printed (message on stderr, exit status 1).

3. is_safe() copies Available into Work and scans the processes in index order in repeated passes; any unfinished process with Need <= Work finishes and returns its Allocation to Work. It reports SAFE with the finishing order, or UNSAFE when a full pass finishes nobody. It never modifies the state.

4. try_request() checks Request > Need first, then Request > Available, then tentatively grants the request, runs is_safe(), and either commits (GRANTED) or undoes the tentative grant exactly (DENIED: would lead to unsafe state).

5. Helpers return status codes (REQ_GRANTED, REQ_EXCEEDS_MAX, REQ_INSUFFICIENT, REQ_UNSAFE, REQ_ERROR) and never call exit(); main() turns the codes into the required messages.

6. Compiled with the exact required flags; the program and all test inputs were also run under AddressSanitizer, UndefinedBehaviorSanitizer and LeakSanitizer with no reports.

CODE (bankers.c):

```
/*
 * bankers.c  --  Banker's Algorithm simulator (deadlock avoidance)
 *
 * Reads a system state (Available, Allocation, Max) from stdin, then answers
 * a list of queries:
 *     S                       safety check
 *     R i r0 r1 ... r(M-1)    resource request from process i
 *
 * Build : gcc -std=c99 -Wall -Wextra -Werror -pedantic -o bankers bankers.c
 * Run   : ./bankers < input.txt
 *
 * Helpers never call exit(); they return status codes and main() decides
 * what to do, so every error path can free its memory.
 */
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#define MAX_N   20
#define MAX_M   10
#define MAX_Q   100
#define MAX_VAL 1000

/* status codes returned by try_request() */
#define REQ_GRANTED      0
#define REQ_EXCEEDS_MAX  1
#define REQ_INSUFFICIENT 2
#define REQ_UNSAFE       3
#define REQ_ERROR       (-1)

/* ---------------------------------------------------------------- */
/* Matrix helpers                                                   */
/* ---------------------------------------------------------------- */

void free_matrix(int **m, int rows)
{
    if (m == NULL) return;
    for (int r = 0; r < rows; r++) free(m[r]);   /* free(NULL) is harmless */
    free(m);
}

/* rows x cols matrix of zeros, or NULL if any allocation fails */
int **alloc_matrix(int rows, int cols)
{
    int **mat = calloc((size_t)rows, sizeof(int *));
    if (mat == NULL) return NULL;
    for (int r = 0; r < rows; r++) {
        mat[r] = calloc((size_t)cols, sizeof(int));
        if (mat[r] == NULL) {
            free_matrix(mat, rows);
            return NULL;
        }
    }
    return mat;
}

/* ---------------------------------------------------------------- */
/* Core algorithms                                                  */
/* ---------------------------------------------------------------- */

/*
 * Need = Max - Allocation.
 * Returns 0 on success. If some Allocation[i][j] > Max[i][j] the state is
 * invalid and the function returns (i * m + j) + 1 so main() can report
 * exactly which entry is wrong.
 */
int compute_need(int **alloc, int **max, int **need, int n, int m)
{
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (alloc[i][j] > max[i][j]) return i * m + j + 1;
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
    return 0;
}

/*
 * Safety algorithm.
```

```
 * Work starts as a copy of Available. Repeatedly scan the processes in
 * index order; an unfinished process whose Need <= Work can run to
 * completion, so it returns its Allocation to Work and is appended to the
 * safe sequence. Stop when everything has finished (safe) or a full pass
 * finishes nobody (unsafe).
 *
 * Returns 1 = safe (sequence stored in seq_out if it is not NULL),
 *         0 = unsafe, -1 = out of memory.
 * The state (alloc, need, avail) is never modified.
 */
int is_safe(int **alloc, int **need, int *avail, int n, int m, int *seq_out)
{
    int *work = malloc((size_t)m * sizeof(int));
    int *finish = calloc((size_t)n, sizeof(int));
    int count = 0, progress = 1;

    if (work == NULL || finish == NULL) {
        free(work);
        free(finish);
        return -1;
    }
    memcpy(work, avail, (size_t)m * sizeof(int));

    while (count < n && progress) {
        progress = 0;
        for (int i = 0; i < n; i++) {
            int can_run = 1;
            if (finish[i]) continue;
            for (int j = 0; j < m; j++) {
                if (need[i][j] > work[j]) { can_run = 0; break; }
            }
            if (can_run) {
                for (int j = 0; j < m; j++) work[j] += alloc[i][j];
                finish[i] = 1;
                if (seq_out != NULL) seq_out[count] = i;
                count++;
                progress = 1;
            }
        }
    }

    free(work);
    free(finish);
    return count == n;
}

/*
 * Resource-request algorithm for process i.
 *   1. req > Need[i]      -> REQ_EXCEEDS_MAX   (checked first)
 *   2. req > Available    -> REQ_INSUFFICIENT
 *   3. otherwise pretend to grant, run the safety check:
 *        safe   -> keep the new state, REQ_GRANTED
 *        unsafe -> undo the pretend grant exactly, REQ_UNSAFE
 * *granted is set to 1 only when the request is committed.
 * Returns REQ_ERROR on out-of-memory (state is restored first).
 */
int try_request(int **alloc, int **need, int *avail, int i, const int *req,
                int n, int m, int *granted)
{
    int safe;

    *granted = 0;
```

```
    for (int j = 0; j < m; j++)
        if (req[j] > need[i][j]) return REQ_EXCEEDS_MAX;
    for (int j = 0; j < m; j++)
        if (req[j] > avail[j]) return REQ_INSUFFICIENT;

    /* tentative grant */
    for (int j = 0; j < m; j++) {
        avail[j]    -= req[j];
        alloc[i][j] += req[j];
        need[i][j]  -= req[j];
    }

    safe = is_safe(alloc, need, avail, n, m, NULL);
    if (safe == 1) {
        *granted = 1;
        return REQ_GRANTED;
    }

    /* rollback: put the state back exactly as it was */
    for (int j = 0; j < m; j++) {
        avail[j]    += req[j];
        alloc[i][j] -= req[j];
        need[i][j]  += req[j];
    }
    return (safe == 0) ? REQ_UNSAFE : REQ_ERROR;
}

/* ------------------------------------------------------------------ */
/* Input parsing                                                       */
/* ------------------------------------------------------------------ */

/* Reads one whole-token integer in [lo, hi]; prints an error and returns -1 on failure. */
static int read_int(int *out, long lo, long hi, const char *what)
{
    char tok[64];
    char *end;
    long v;

    if (scanf("%63s", tok) != 1) {
        fprintf(stderr, "Error: input ended early, missing %s\n", what);
        return -1;
    }
    errno = 0;
    v = strtol(tok, &end, 10);
    if (*end != '\0' || errno == ERANGE) {
        fprintf(stderr, "Error: '%s' is not a valid integer (%s)\n", tok, what);
        return -1;
    }
    if (v < lo || v > hi) {
        fprintf(stderr, "Error: %s = %ld is out of range [%ld, %ld]\n", what, v, lo, hi);
        return -1;
    }
    *out = (int)v;
    return 0;
}

/* Reads an n x m matrix of values in [0, MAX_VAL]. */
static int read_matrix(int **mat, int n, int m, const char *name)
{
    char what[64];
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
            sprintf(what, "%s[%d][%d]", name, i, j);
            if (read_int(&mat[i][j], 0, MAX_VAL, what) < 0) return -1;
        }
    }
    return 0;
}

/* ---------------------------------------------------------------- */
/* main                                                             */
/* ---------------------------------------------------------------- */

int main(void)
{
    int n = 0, m = 0, q = 0;
    int **alloc = NULL, **max = NULL, **need = NULL;
    int *avail = NULL, *seq = NULL;
    int qtype[MAX_Q];              /* 0 = safety check, 1 = request */
    int qproc[MAX_Q];
    int qreq[MAX_Q][MAX_M];
    int status = EXIT_FAILURE;
    char extra[64];
    int bad;

    /* ---- read and validate everything before printing anything ---- */
    if (read_int(&n, 1, MAX_N, "N") < 0) goto cleanup;
    if (read_int(&m, 1, MAX_M, "M") < 0) goto cleanup;

    alloc = alloc_matrix(n, m);
    max   = alloc_matrix(n, m);
    need  = alloc_matrix(n, m);
    avail = calloc((size_t)m, sizeof(int));
    seq   = calloc((size_t)n, sizeof(int));
    if (!alloc || !max || !need || !avail || !seq) {
        fprintf(stderr, "Error: out of memory\n");
        goto cleanup;
    }

    for (int j = 0; j < m; j++) {
        char what[64];
        sprintf(what, "Available[%d]", j);
        if (read_int(&avail[j], 0, MAX_VAL, what) < 0) goto cleanup;
    }
    if (read_matrix(alloc, n, m, "Allocation") < 0) goto cleanup;
    if (read_matrix(max, n, m, "Max") < 0) goto cleanup;

    bad = compute_need(alloc, max, need, n, m);
    if (bad != 0) {
        fprintf(stderr, "Error: Allocation[%d][%d] is greater than Max[%d][%d]\n",
                (bad - 1) / m, (bad - 1) % m, (bad - 1) / m, (bad - 1) % m);
        goto cleanup;
    }

    if (read_int(&q, 0, MAX_Q, "Q") < 0) goto cleanup;
    for (int k = 0; k < q; k++) {
        char tok[64];
        if (scanf("%63s", tok) != 1) {
            fprintf(stderr, "Error: input ended early, expected query %d of %d\n", k + 1, q);
            goto cleanup;
        }
        if (strcmp(tok, "S") == 0) {
```

```
            qtype[k] = 0;
        } else if (strcmp(tok, "R") == 0) {
            qtype[k] = 1;
            if (read_int(&qproc[k], 0, n - 1, "process index of request") < 0) goto cleanup;
            for (int j = 0; j < m; j++) {
                if (read_int(&qreq[k][j], 0, MAX_VAL, "request amount") < 0) goto cleanup;
            }
        } else {
            fprintf(stderr, "Error: unknown query '%s' (expected S or R)\n", tok);
            goto cleanup;
        }
    }
    if (scanf("%63s", extra) == 1) {
        fprintf(stderr, "Error: unexpected extra input '%s' after the last query\n", extra);
        goto cleanup;
    }

    /* ---- answer the queries ---- */
    for (int k = 0; k < q; k++) {
        if (qtype[k] == 0) {
            int r = is_safe(alloc, need, avail, n, m, seq);
            if (r < 0) { fprintf(stderr, "Error: out of memory\n"); goto cleanup; }
            if (r == 1) {
                printf("SAFE:");
                for (int i = 0; i < n; i++) printf(" %d", seq[i]);
                printf("\n");
            } else {
                printf("UNSAFE\n");
            }
        } else {
            int granted;
            int r = try_request(alloc, need, avail, qproc[k], qreq[k], n, m, &granted);
            if (r == REQ_ERROR) { fprintf(stderr, "Error: out of memory\n"); goto cleanup; }
            if (r == REQ_GRANTED)           printf("GRANTED\n");
            else if (r == REQ_EXCEEDS_MAX)  printf("DENIED: exceeds maximum claim\n");
            else if (r == REQ_INSUFFICIENT) printf("DENIED: insufficient resources\n");
            else                            printf("DENIED: would lead to unsafe state\n");
        }
        printf("\n");
    }
    status = EXIT_SUCCESS;

cleanup:
    free_matrix(alloc, n);
    free_matrix(max, n);
    free_matrix(need, n);
    free(avail);
    free(seq);
    return status;
}
```

COMPILATION:

```
aditya-forge@LAPTOP-184D82EB:~/bankers$ gcc -std=c99 -Wall -Wextra -Werror -pedantic -o bankers
bankers.c
```

OUTPUT:

Test 1: the sample from the assignment

Expected output from the assignment: SAFE: 1 3 4 0 2, then GRANTED.

```
aditya-forge@LAPTOP-184D82EB:~/bankers$ cat tests/test1_sample.txt
5 3
3 3 2
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
2
S
R 1 1 0 2
aditya-forge@LAPTOP-184D82EB:~/bankers$ ./bankers < tests/test1_sample.txt
SAFE: 1 3 4 0 2

GRANTED
```

Test 2: a safe system granting a valid request

4 processes, 2 resources. P3 asks for (1 1), which fits its Need and Available and leaves the system safe, so it is GRANTED; the second safety check shows the updated state.

```
aditya-forge@LAPTOP-184D82EB:~/bankers$ cat tests/test2_safe_grant.txt
4 2
2 1
1 0
0 1
1 1
0 0
3 2
2 2
2 2
1 1
3
S
R 3 1 1
S
aditya-forge@LAPTOP-184D82EB:~/bankers$ ./bankers < tests/test2_safe_grant.txt
SAFE: 1 2 3 0

GRANTED

SAFE: 3 1 2 0
```

Test 3: a request that exceeds the claim

P0 asks for 8 units of resource 0 but its Need is 7 and P2 asks for 5 units of resource 2 but its Need is 0. Both are DENIED: exceeds maximum claim, and the final safety check shows the state did not change.

```
aditya-forge@LAPTOP-184D82EB:~/bankers$ cat tests/test3_exceeds_claim.txt
5 3
```

```
3 3 2
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
3
R 0 8 0 0
R 2 0 0 5
S
aditya-forge@LAPTOP-184D82EB:~/bankers$ ./bankers < tests/test3_exceeds_claim.txt
DENIED: exceeds maximum claim

DENIED: exceeds maximum claim

SAFE: 1 3 4 0 2
```

Test 4: a request that would make the system unsafe

After P1's request is granted, P4 asking for (3 3 0) is more than Available, and P0 asking for (0 2 0) fits Available but would leave no safe sequence, so it is rolled back. The last query shows the state is the same as before that request.

```
aditya-forge@LAPTOP-184D82EB:~/bankers$ cat tests/test4_unsafe_request.txt
5 3
3 3 2
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
4
R 1 1 0 2
R 4 3 3 0
R 0 0 2 0
S
aditya-forge@LAPTOP-184D82EB:~/bankers$ ./bankers < tests/test4_unsafe_request.txt
GRANTED

DENIED: insufficient resources

DENIED: would lead to unsafe state

SAFE: 1 3 4 0 2
```

Extra tests

Extra: all-zero request in a safe state

A request of all zeros is legal, passes the safety check and changes nothing.

```
aditya-forge@LAPTOP-184D82EB:~/bankers$ cat tests/test5_zero_request.txt
5 3
3 3 2
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
3
R 2 0 0 0
S
R 2 0 0 0
aditya-forge@LAPTOP-184D82EB:~/bankers$ ./bankers < tests/test5_zero_request.txt
GRANTED

SAFE: 1 3 4 0 2

GRANTED
```

Extra: an initial state that is already unsafe

No process can finish with Available = (1 1), so the safety check prints UNSAFE and any request is denied.

```
aditya-forge@LAPTOP-184D82EB:~/bankers$ cat tests/test6_unsafe_state.txt
3 2
1 1
1 0
0 1
0 0
3 3
3 3
3 3
2
S
R 2 0 0
aditya-forge@LAPTOP-184D82EB:~/bankers$ ./bankers < tests/test6_unsafe_state.txt
UNSAFE

DENIED: would lead to unsafe state
```

Invalid input is rejected

Each file makes the program print a message on stderr, print nothing on stdout and exit with status 1.

Input file	Exit	Message on stderr
bad1_alloc_exceeds_max.txt	1	Error: Allocation[0][0] is greater than Max[0][0]
bad2_negative_available.txt	1	Error: Available[0] = -1 is out of range [0, 1000]
bad3_too_few_tokens.txt	1	Error: input ended early, missing Q
bad4_extra_tokens.txt	1	Error: unexpected extra input 'S' after the last query
bad5_unknown_query.txt	1	Error: unknown query 'X' (expected S or R)
bad6_process_out_of_range.txt	1	Error: process index of request = 5 is out of range [0, 1]
bad7_zero_processes.txt	1	Error: N = 0 is out of range [1, 20]
bad8_non_numeric.txt	1	Error: '1.5' is not a valid integer (Available[0])
bad9_negative_request.txt	1	Error: request amount = -1 is out of range [0, 1000]

Why rollback on a failed request is essential

A request is checked for safety only after the state has been changed, so Allocation, Need and Available already contain the tentative grant when the safety algorithm runs. If the result is unsafe and those changes are not undone, the program keeps a state it has just shown can lead to deadlock, and the resources stay recorded as given to the process even though the request was reported as DENIED. Every later query would then be answered from a wrong state: safe requests could be refused, unsafe ones accepted, and printed safe sequences would be invalid. Rolling back restores Available, Allocation and Need exactly, so a denied request has no side effects and the system stays in the last state known to be safe. Test 4 shows this: the safety check after the denied request still gives SAFE: 1 3 4 0 2.

Memory and compilation checks

The program compiles with the required flags with no warnings. All 15 test inputs, valid and invalid, were run under AddressSanitizer, UndefinedBehaviorSanitizer and LeakSanitizer with no invalid accesses, no undefined behaviour and no leaks.